SkillMatch – Dossier technique

« On recrute sur les compétences, pas sur le CV »

ESPRIT • 4DAS I13

Validation intermédiaire – Séance 4

Année universitaire 2026–2027

Architecture Web Distribuée

ÉQUIPE

Chahd Najmaoui

Sarra BenBoubaker

Skander Selmi

Koussay Laifi

Amir Khemissi

Meriem Atia

ENCADRANT

M. Oussema Sellami

1. Résumé et périmètre

SkillMatch est une plateforme de recrutement fondée sur les compétences vérifiées plutôt que sur la seule lecture des CV. Les candidats construisent leur profil, passent des tests techniques et postulent ; les recruteurs publient des offres et suivent les candidatures ; un administrateur gère les comptes, le catalogue de tests et la modération.

L'application est conçue comme **6 microservices métier + 1 microservice commun** (Auth + Notifications), derrière une API Gateway. Il n'existe pas de microservice « User » : l'identité fait partie du commun.

Ce document décrit une **conception prévisionnelle** : les composants, API et échanges décrits ne sont pas encore intégrés. L'objectif de la séance 4 est un design cohérent, un dépôt Git propre et une répartition réaliste du travail.

Livrable Blackboard	Emplacement
Dépôt Git public	https://github.com/skander101/SkillMatch-4dasI13 (enseignant ajouté comme collaborateur)
Dossier technique	Ce document (versions PDF et Word dans Documentation/)
Présentation séance 4	Documentation/Presentation/
Architecture globale + autres diagrammes	Documentation/Diagrams/

2. Problématique, objectifs et ODD

Contexte. Les recrutements classiques s'appuient sur les diplômes et le CV, qui ne prouvent pas toujours les aptitudes techniques réelles.

Problématique. Comment garantir une mise en relation plus juste et plus efficace entre candidats et recruteurs en privilégiant les compétences réelles, avec une architecture distribuée modulaire et évolutive ?

Objectifs.

- Permettre au candidat de créer son profil, gérer ses CV et passer des tests techniques.
- Aider le recruteur à identifier les candidats qualifiés et suivre les candidatures.
- Recommander des offres adaptées aux compétences et badges du candidat.
- Simplifier la gestion des offres et candidatures, et faciliter la communication (messagerie, alertes, notifications).

Scénario. Lina maîtrise Java et SQL, sans diplôme prestigieux. Elle complète son profil, réussit le test « Java backend » et obtient un badge. Une offre correspondante est publiée : son alerte se déclenche, elle postule, le recruteur la présélectionne et planifie un entretien ; la messagerie accompagne les échanges.

ODD visés (hypothèse : à ajuster avec votre storytelling).

ODD	Lien avec le projet	Services concernés
8 – Travail décent	Faciliter l'accès à l'emploi	Offres & Candidatures, Alertes & Favoris
4 – Éducation de qualité	Valoriser et certifier les compétences	Tests & Compétences, Profils & CV
10 – Inégalités réduites	Sélection sur compétences, pas sur le diplôme	Offres (matching), Tests
16 – Institutions efficaces et transparentes	Avis modérés, accès contrôlés	Entreprises & Avis, Commun

Hors périmètre initial : moteur d'IA autonome de matching, paiement, plateforme complète dès la semaine 4.

3. Acteurs et cas d·utilisation

Acteur	Responsabilités principales
Candidat	S’inscrire, créer son profil et ses CV, passer des tests, consulter les offres recommandées, postuler, suivre ses candidatures, gérer alertes/favoris, échanger, publier un avis.
Recruteur	Créer son entreprise, publier des offres, consulter les candidatures, vérifier les compétences, présélectionner, planifier les entretiens, décider.
Administrateur	Gérer les comptes et rôles, gérer le catalogue de tests, modérer les avis.

Le diagramme de cas d’utilisation général (Documentation/Diagrams/use-case-general) regroupe six grands cas : gérer les offres, les candidatures, le profil et les CV, les tests, les interactions (messages, alertes) et l’entreprise ; plus l’inscription/connexion et l’administration.

Règles transverses. La consultation des offres et des entreprises est publique ; toute opération personnelle requiert un compte actif. Chaque service vérifie le rôle et les droits sur la ressource (propriétaire d’une offre, auteur d’un avis, participant d’une conversation).

4. Fonctionnalités et règles métier

Domaine	Fonctions
Profil & CV	Compétences, expériences, badges, complétude, dépôt de CV PDF.
Tests	Catalogue, passage, correction automatique, résultats, 3 tentatives max.
Offres & candidatures	Recherche, recommandation, publication, dépôt, suivi, entretien, décision.
Entreprises & avis	Fiche entreprise, avis modéré, note moyenne.
Alertes & favoris	Critères enregistrés, notification sur nouvelle offre, sauvegarde.
Messagerie	Conversations liées à une candidature, historique, lecture.
Commun	Inscription, connexion JWT, gestion des rôles, notifications.

Règles métier clés (une par service). Candidature unique par candidat et offre (409) • CV PDF de 5 Mo maximum • 3 tentatives de test maximum, aucune après réussite • un avis n'est public qu'après modération • 5 alertes actives maximum par candidat • seuls les participants lisent une conversation • notification lisible uniquement par son destinataire.

5. Architecture globale

Le client Angular appelle uniquement la **Spring Cloud Gateway**, qui valide le JWT et route vers les services. **Eureka** assure la découverte et **Config Server** centralise la configuration. Les appels synchrones utilisent REST (OpenFeign) ; les événements métier transitent par **RabbitMQ** (exchange topic `skillmatch.events`). Chaque service possède sa propre base MySQL (XAMPP, un serveur, 7 schémas).

Flux : Angular → Gateway (JWT) → {Commun, Offres, Profils, Entreprises, Tests, Alertes, Messagerie} ; services ↔ RabbitMQ ; chaque service → sa base MySQL. Le schéma complet est fourni dans Documentation/Diagrams/architecture-globale (fichier image/PDF indépendant).

Élément	Rôle
Angular	Interface unique pour les trois acteurs.
API Gateway	Point d'entrée, validation JWT, routage, préfixe `/api`.
Eureka + Config Server	Annuaire des services et configuration externalisée.
7 microservices Spring Boot	Domaines autonomes, API REST, traitement d'événements.
RabbitMQ	Événements asynchrones découplés.
MySQL / XAMPP	`commun_db`, `offres_db`, `profils_db`, `entreprises_db`, `tests_db`, `alertes_db`, `messagerie_db`.

6. Sécurité

L'authentification est portée par le **service commun (Auth, JWT)**.

1. `POST /api/auth/login` : Commun vérifie l'email et le mot de passe (hash BCrypt) puis émet un JWT signé contenant `sub` (userId), `role` et `exp` (durée 1 h, hypothèse).
2. Le client envoie `Authorization: Bearer <jwt>` à la Gateway.
3. La Gateway contrôle la signature et l'expiration, distingue routes publiques et protégées, puis transmet `X-User-Id` et `X-User-Role` aux services.
4. Chaque service applique les droits sur la ressource (propriétaire, auteur, participant).

La clé de signature et les secrets sont externalisés (Config Server, variables d'environnement) et jamais commités. Les réponses d'erreur n'exposent aucune trace technique. Les services internes ne sont pas exposés directement : seule la Gateway est publique.

7. Technologies

Couche	Technologie
Frontend	Angular
Backend	Java 17, Spring Boot 3, Spring Data JPA
Routage / distribution	Spring Cloud Gateway, Eureka, Config Server, OpenFeign
Sécurité	JWT (émis par le service commun), BCrypt
Données	MySQL sur XAMPP, un schéma par service
Asynchrone	RabbitMQ
Qualité / livraison	Swagger/OpenAPI, Postman, JUnit, Docker, Docker Compose, GitHub Actions
Versionnement	Git, GitHub (Projects / Issues)

8. Découpage des microservices et données

Chaque service est propriétaire de sa base. Entre services, seuls des **identifiants** (Long/BIGINT) sont référencés : ni clé étrangère, ni jointure, ni requête inter-bases. Les listes sans cycle de vie propre (compétences, expériences, badges, questions, participants) sont des colonnes JSON ; les valeurs fixes sont des enums Java ; les valeurs calculées (complétude, score de matching) ne sont pas stockées.

Service	Responsable	Base	Entités
Commun (Auth + Notifications)	Toute l'équipe	commun_db	Utilisateur, Notification
Offres & Candidatures	Chahd	offres_db	Offre, Candidature
Profils & CV	Sarra	profils_db	Profil, CV
Entreprises & Avis	Skander	entreprises_db	Entreprise, AvisEntreprise
Tests & Compétences	Koussay	tests_db	TestTechnique, ResultatTest

Service	Responsable	Base	Entités
Alertes & Favoris	Amir	alertes_db	AlerteEmploi, OffreSauvegardee
Messagerie	Meriem	messagerie_db	Conversation, Message

Compromis. Ce découpage favorise la responsabilité unique, le développement parallèle et la traçabilité des contributions ; son coût est la coordination des échanges, maîtrisée par des contrats REST figés et un format d'événement commun.

9. Conception détaillée des microservices

Pour chaque service : entités, endpoints principaux (accès et codes de réponse), interactions, règle métier. Les diagrammes de classes figurent dans Documentation/Diagrams/. Les endpoints internes (appelés par un autre service) sont marqués *interne*.

9.1 Commun : Auth + Notifications (toute l'équipe)

Entités. `Utilisateur` (id, nom, email unique, motDePasseHash, role : CANDIDAT/RECRUTEUR/ADMIN, actif) ; `Notification` (id, userId, type, message, canal : IN_APP/EMAIL, lu, date). Relation 1 → N.

Endpoint	Accès	Codes
POST /auth/register	Public	201, 400, 409
POST /auth/login	Public	200, 401
GET /users/me	Connecté	200, 401
GET /users	ADMIN	200, 403
PUT /users/{id}/role	ADMIN	200, 400, 403, 404
PUT /users/{id}/active	ADMIN	200, 403, 404
GET /notifications	Connecté	200, 401
PUT /notifications/{id}/read	Destinataire	200, 403, 404
PUT /notifications/read-all	Connecté	200

Événements consommés : `StatusChanged`, `AlertMatched`, `TestPassed`, `MessageSent` (création d'une notification in-app ou email). Aucun appel synchrone sortant.

Règle : une notification n'est lisible que par son destinataire. Remarque : *Alerte* = règle configurée par le candidat (service Alertes) ; *Notification* = message envoyé quand un événement survient (service Commun).

9.2 Offres & Candidatures (Chahd)

Entités. `Offre` (id, recruteurId, entrepriseId, testRequisId, titre, description, competences JSON, localisation, contrat : CDI/CDD/STAGE/FREELANCE, salaire, dateExpiration, statut : BROUILLON/PUBLIEE/FERMEE/EXPIREE) ; `Candidature` (id, offreId, userId, cvId, lettre, statut, dateEntretien, noteRecruteur, commentaireRecruteur). Relation 1 → N, UNIQUE(offreId, userId). Le matching n'est pas un service ; l'entretien est un statut + une date.

Endpoint	Accès	Codes
GET /offers	Public	200
GET /offers/{id}	Public	200, 404
GET /offers/recommended	CANDIDAT	200, 401

Endpoint	Accès	Codes
POST /offers	RECRUTEUR	201, 400, 403
PUT /offers/{id}	Propriétaire	200, 400, 403, 404
POST /applications	CANDIDAT	201, 403, 404, 409, 422
GET /applications/me	CANDIDAT	200
GET /offers/{id}/applications	Propriétaire	200, 403
PUT /applications/{id}/status	RECRUTEUR / CANDIDAT (retrait)	200, 403, 409

Internes : `GET /applications/{id}` , `GET /applications/exists` (utilisés par Messagerie et Entreprises).

Recommandation. `GET /offers/recommended` calcule en code un score pondéré (hypothèse : 60 % compétences communes, 30 % badges correspondants, 10 % localisation) à partir du profil obtenu auprès de Profils.

Cycle de la candidature. RECUE → PRESELECTIONNEE → ENTRETIEN_PLANIFIE → ACCEPTEE ; REFUSEE possible à chaque étape active ; RETIREE par le candidat avant décision.

Interactions. Sync : Profils (profil, CV) et Tests (réussite du test requis). Publie `OfferPublished` , `ApplicationSubmitted` , `StatusChanged` .

Règles : une candidature par couple candidat/offre ; CV existant ; test requis réussi ; offre expirée refusée (422) ; transitions de statut contrôlées (409).

9.3 Profils & CV (Sarra)

Entités. `Profil` (id, userId unique, titre, localisation, competences JSON, experiences JSON, badges JSON ; complétude calculée) ; `CV` (id, profilId, fichierUri, dateUpload). Relation 1 → N.

Endpoint	Accès	Codes
POST /profiles	CANDIDAT	201, 400, 409
GET /profiles/me	CANDIDAT	200, 404
PUT /profiles/me	CANDIDAT	200, 400

Endpoint	Accès	Codes
GET /profiles/me/completeness	CANDIDAT	200, 404
POST /profiles/me/cv	CANDIDAT	201, 400, 413
GET /profiles/me/cv	CANDIDAT	200
DELETE /profiles/me/cv/{cvId}	CANDIDAT	204, 403, 404
GET /profiles/{userId}	RECRUTEUR, interne	200, 403, 404
GET /profiles/{userId}/cv/{cvId}	interne	200, 404

Interactions. Fournit ses données à Offres (appels REST entrants). Consomme `TestPassed` (ajout d'un badge) ; publie `ProfileUpdated`.

Règles : un profil par candidat ; CV en PDF, 5 Mo maximum ; complétude recalculée à chaque modification.

9.4 Entreprises & Avis (Skander)

Entités. `Entreprise` (id, recruteurId unique, nom, secteur, siteWeb, logo, description, noteMoyenne) ; `AvisEntreprise` (id, entrepriseId, userId, note 1–5, titre, commentaire, statut : EN_ATTENTE/PUBLIE/REJETE, date). Relation 1 → N, UNIQUE(entrepriseId, userId).

Endpoint	Accès	Codes
POST /companies	RECRUTEUR	201, 400, 409
GET /companies	Public	200
GET /companies/{id}	Public	200, 404
PUT /companies/{id}	Propriétaire	200, 403, 404
POST /companies/{id}/reviews	CANDIDAT	201, 403, 409, 422
GET /companies/{id}/reviews	Public (avis publiés)	200, 404
GET /reviews/pending	ADMIN	200, 403
PUT /reviews/{id}/status	ADMIN	200, 403, 409
DELETE /reviews/{id}	Auteur / ADMIN	204, 403

Interactions. Sync : Offres (`GET /applications/exists`) pour vérifier qu'un candidat a bien postulé chez l'entreprise avant d'autoriser un avis. Aucun événement publié ou consommé (hypothèse : service volontairement léger côté asynchrone).

Règles : une entreprise par recruteur ; un avis par candidat et entreprise ; `noteMoyenne` recalculée (moyenne des avis PUBLIE) après chaque modération.

9.5 Tests & Compétences (Koussay)

Entités. `TestTechnique` (id, titre, competence, dureeMinutes, seuil, questions JSON avec indexBonneReponse) ; `ResultatTest` (id, testId, userId, score, tentative, reussi, date). Relation 1 → N.

Endpoint	Accès	Codes
POST /tests	ADMIN	201, 400, 403
GET /tests	Connecté	200
GET /tests/{id}	Connecté (sans solutions)	200, 404
PUT /tests/{id}	ADMIN	200, 400, 403, 404
DELETE /tests/{id}	ADMIN	204, 403, 404

Endpoint	Accès	Codes
POST /tests/{id}/submit	CANDIDAT	200, 400, 404, 409
GET /results/me	CANDIDAT	200
GET /tests/{id}/results	ADMIN	200, 403
GET /results/check	interne	200, 404

Interactions. Appelé en REST par Offres (`/results/check`). Publie `TestPassed` au premier franchissement du seuil.

Règles : correction automatique ; 3 tentatives maximum (409 au-delà) ; aucune tentative après réussite ; solutions jamais exposées.

9.6 Alertes & Favoris (Amir)

Entités. `AlerteEmploi` (id, userId, motsCles JSON, localisation, contrat, frequence, active) ; `OffreSauvegardee` (id, userId, offerId, date), UNIQUE(userId, offerId).

Endpoint	Accès	Codes
POST /alerts	CANDIDAT	201, 400, 409, 422
GET /alerts/me	CANDIDAT	200
GET /alerts/{id}	Propriétaire	200, 403, 404
PUT /alerts/{id}	Propriétaire	200, 403, 404
PUT /alerts/{id}/active	Propriétaire	200, 403, 422
DELETE /alerts/{id}	Propriétaire	204, 403
POST /favorites	CANDIDAT	201, 404, 409, 422
GET /favorites/me	CANDIDAT	200
DELETE /favorites/{id}	Propriétaire	204, 403

Interactions. Sync : Offres (`GET /offers/{id}`) pour vérifier qu'une offre est PUBLIEE avant mise en favori. Consomme `OfferPublished` ; publie `AlertMatched` pour chaque alerte active correspondante (mots-clés, localisation, contrat).

Règles : 5 alertes actives maximum par candidat (422) ; un favori par offre ; seule une offre publiée peut être sauvegardée.

9.7 Messagerie (Meriem)

Entités. `Conversation` (id, candidatureId, participantIds JSON, dernierMessageAt) ; `Message` (id, conversationId, senderId, contenu, date, lu). Relation 1 → N.

Endpoint	Accès	Codes
POST /conversations	Participant	201, 403, 409, 422
GET /conversations/me	Connecté	200
GET /conversations/{id}	Participant	200, 403, 404
GET /conversations/{id}/messages	Participant (paginé)	200, 403
POST /conversations/{id}/messages	Participant	201, 400, 403
PUT /conversations/{id}/read	Participant	200, 403
GET /conversations/{id}/unread-count	Participant	200, 403
PUT /messages/{id}	Auteur (non lu)	200, 403, 409
DELETE /messages/{id}	Auteur	204, 403

Interactions. Sync : Offres (`GET /applications/{id}`) pour vérifier la candidature et ses participants à l'ouverture. Publie `MessageSent`.

Règles : accès limité aux participants ; message non vide ; conversation liée à une candidature valide.

10. Communications synchrones (REST)

Un appel synchrone n'est utilisé que lorsqu'un service a besoin d'une information avant de finaliser son action. Les appels passent par OpenFeign et les noms résolus par Eureka ; timeouts et repli seront configurés au développement.

Appelant → appelé	Ressource interne	Justification
Offres → Profils	GET /profiles/{userId}	Compétences et localisation pour le score de recommandation.
Offres → Tests	GET /results/check	Vérifier la réussite du test requis avant candidature.
Alertes → Offres	GET /offers/{id}	Vérifier que l'offre est publiée avant favori.
Messagerie → Offres	GET /applications/{id}	Vérifier candidature et participants.
Entreprises → Offres	GET /applications/exists	Autoriser un avis uniquement après candidature.

Scénario : dépôt de candidature. Candidat → POST /applications → Offres vérifie le CV (Profils) et le test requis (Tests) → contrôles métier (doublon, offre expirée) → création → 201 → événement `ApplicationSubmitted`.

Erreurs : CV introuvable → 404 ; test non réussi → 403 ; doublon → 409 ; offre expirée → 422 ; service appelé indisponible → 503 (jamais assimilé à un refus métier). Pour la recommandation, un profil absent donne une liste d'offres récentes sans score.

11. Communications asynchrones (RabbitMQ)

Exchange `skillmatch.events` (topic), une file par consommateur.

Événement	Émetteur	Consommateur : effet
OfferPublished	Offres	Alertes : comparaison aux alertes actives
AlertMatched	Alertes	Commun : notification du candidat
StatusChanged	Offres	Commun : notification du candidat
TestPassed	Tests	Profils : badge ; Commun : notification
MessageSent	Messagerie	Commun : notification du destinataire
ApplicationSubmitted	Offres	Disponible pour extension (notification recruteur)

Événement	Émetteur	Consommateur : effet
ProfileUpdated	Profils	Disponible pour extension (recalcul de recommandations)

Format. Chaque message porte `eventId`, `type`, `occurredAt`, `source`, `data`. L'`eventId` garantit un traitement idempotent ; les échecs sont gérés par accusé de réception, nouvelle tentative et journalisation. La publication d'une offre ne dépend pas de la disponibilité de Notifications.

Scénario : nouvelle offre. Le recruteur publie → Offres émet `OfferPublished` → Alertes évalue les critères → `AlertMatched` → Commun crée la notification du candidat.

12. Conventions d·API

Routes publiques sous `/api/...` via la Gateway ; JSON en entrée et sortie ; erreurs au format `application/problem+json` ; documentation OpenAPI/Swagger par service.

Code	Signification
200 / 201 / 204	Succès / création / suppression sans corps
400	Données invalides
401 / 403	Non authentifié / accès refusé
404	Ressource introuvable
409	Conflit ou doublon
413	Fichier trop volumineux
422	Règle métier non satisfaite
503	Service dépendant indisponible

Exemple : `409` – type `already-applied`, title « Candidature déjà existante », detail : message métier sans trace technique.

13. Organisation du projet

Répartition. Chaque membre développe un service métier et une pièce du socle commun, ce qui équilibre la charge.

Membre	Service métier	Pièce du socle commun
Chahd Najmaoui	Offres & Candidatures	Auth (JWT)
Sarra BenBoubaker	Profils & CV	Notifications (consommateurs RabbitMQ)
Skander Selmi	Entreprises & Avis	API Gateway
Koussay Laifi	Tests & Compétences	Eureka + Config Server
Amir Khemissi	Alertes & Favoris	Docker Compose + CI GitHub Actions
Meriem Atia	Messagerie	Frontend commun (layout, connexion)

Structure du dépôt `SkillMatch-4dasI13` :

- `Backend/` : offres-service, profils-service, entreprises-service, tests-service, alertes-service, messagerie-service, commun-service, api-gateway, eureka-server, config-server.
- `Frontend/` : application Angular.
- `Documentation/` : `Presentation/`, `Diagrams/`, dossier technique.
- `README.md`.

Gestion du projet. Branches `main` (stable), `develop` (intégration), `feature/<service>-<tâche>` ; fusion par Pull Request relue par un autre membre ; suivi des tâches avec GitHub Projects et Issues. Les commits sont réguliers et atomiques afin de refléter la contribution individuelle.

14. Planning prévisionnel (7 semaines)

Période	Activités
S1 • 16–22 nov. 2026	Conception, Git, diagrammes, dossier technique, présentation séance 4.

Période	Activités
S2 • 23–29 nov.	Socle commun : Auth JWT, Gateway, Eureka, Config, Docker Compose ; début des services.
S3–S4 • 30 nov.–13 déc.	Développement des six services métier et du commun ; frontend (S3–S5).
S5 • 14–20 déc.	Intégration REST et RabbitMQ ; tests de scénarios ; fin du frontend.
S6 • 21–27 déc.	Tests, correctifs, stabilisation.
S7 • 28 déc.–3 janv. 2027	Documentation finale, démonstration, soutenance.

15. Risques et mesures

Risque	Effet	Mesure
Contrats REST divergents	Échec d'intégration	Figer les endpoints, revue par Pull Request, OpenAPI partagé.
Événements incohérents	Notifications absentes	Format d'événement commun, tests d'intégration précoces.
Indisponibilité d'un service	Requête bloquée	Timeout, repli pertinent, réponse 503.
Retard de développement	Blocage collectif	Rôles clairs, binômes de secours, point hebdomadaire.
Sécurité / secrets	Accès non autorisé	JWT, contrôle des rôles, secrets hors dépôt.

16. Conclusion

SkillMatch propose une conception en six microservices métier et un microservice commun, fondée sur des données isolées, des API REST explicites et des événements RabbitMQ. Les responsabilités sont réparties équitablement entre six étudiants. À ce stade, l'objectif est de valider les choix d'architecture, les contrats et l'organisation, non de livrer une application terminée.